Frontend · concept board

TypeScript

Types, interfaces, generics, narrowing and utility types

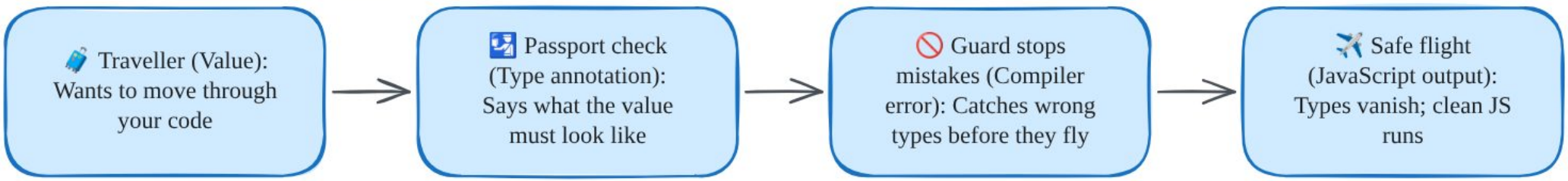
1. Why TypeScript?
2. Basic types, interface vs type
3. Narrowing and discriminated unions
4. Generics
5. Utility types
6. TypeScript with React

Draw on any slide, then Present (arrow keys) or export the board as a PDF.

1. Why TypeScript?

Definition

TypeScript = JavaScript + a static type system. Types are checked at compile time and erased at runtime — the output is plain JavaScript.



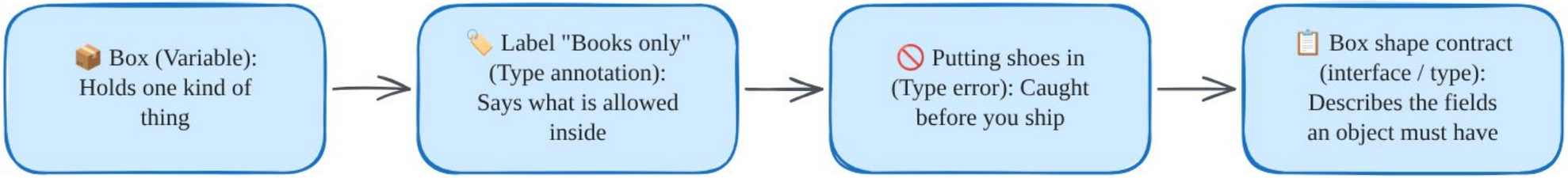
Think of TypeScript like a security checkpoint

- Catches typos, wrong arguments and null bugs before running.
- Better editor autocomplete and safe refactors across a big codebase.
- Types document APIs (props, responses, DB models).

Common mistake

Types don't validate data at runtime. JSON from an API is whatever the server sent — validate it with a schema library such as Zod at the boundary.

2. Basic types, interface vs type



Think of types like labels on storage boxes

Shapes of data

```
interface User {
  id: number
  name: string
  email?: string          // optional
  readonly createdAt: Date
}

type Role = 'admin' | 'doctor' | 'patient'   // union of literals
type Staff = User & { role: Role }           // intersection

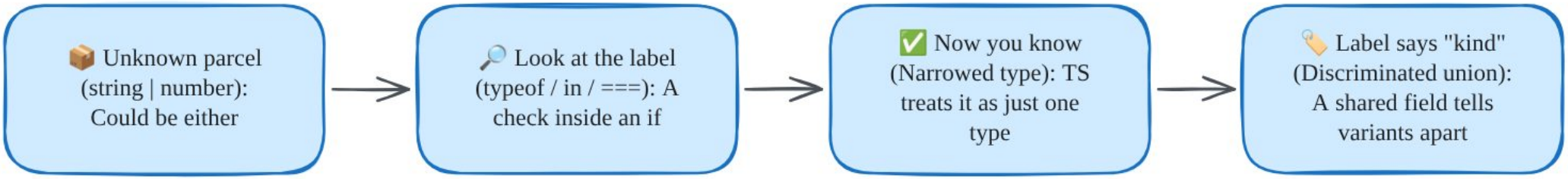
const s: Staff = { id: 1, name: 'Asha', createdAt: new Date(), role: 'doctor' }
const ids: number[] = [1, 2, 3]
const pair: [string, number] = ['age', 31]   // tuple

function greet(u: User): string {
// … 3 more lines in the notes
```

interface	type
Objects and classes; can be extended with extends	Anything: unions, tuples, primitives, mapped types
Declaration merging (re-open to add fields)	Cannot be re-opened
Good default for object shapes and public APIs	Needed for unions and computed types

- any turns type checking off — avoid it.
- unknown is the safe "anything": you must narrow it before use.
- never = a value that can't exist (exhaustive checks, functions that throw).

3. Narrowing and discriminated unions



Think of narrowing like checking a parcel label

Exhaustive switch

```
type Result =
  | { status: 'loading' }
  | { status: 'success'; data: string[] }
  | { status: 'error'; message: string }

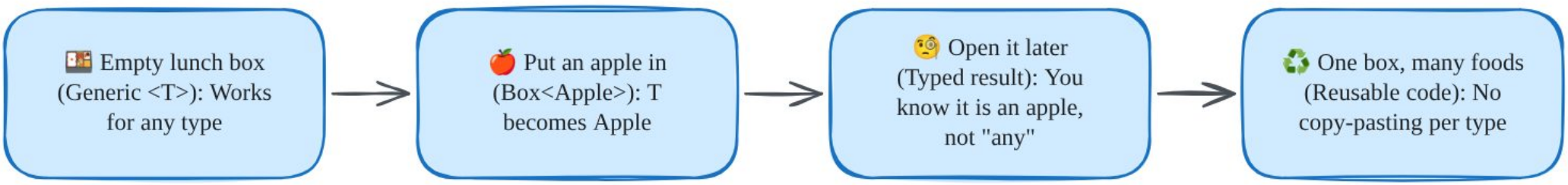
function render(r: Result): string {
  switch (r.status) {
    case 'loading': return 'Loading…'
    case 'success': return r.data.join(', ')   // data exists here
    case 'error':   return 'Error: ' + r.message
    default: {
      const unreachable: never = r              // compile error if a case is missing
      return unreachable
    }
  }
// … 7 more lines in the notes
```

- Narrow with typeof, instanceof, in, equality and truthiness checks.
- A user-defined type guard: function isUser(x: unknown): x is User { … }.

4. Generics

Definition

Generics are type parameters: write one function or type that works for many types while keeping them linked.



Think of generics like a lunch box that fits any food

Generic helpers with constraints

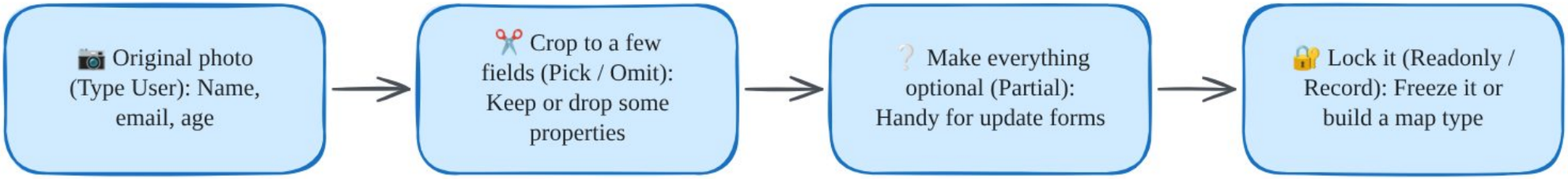
```
function first<T>(items: T[]): T | undefined {
  return items[0]
}

function pluck<T, K extends keyof T>(items: T[], key: K): T[K][] {
  return items.map((item) => item[key])
}

interface ApiResponse<T> { data: T; error: string | null }

const users = [{ id: 1, name: 'Asha' }, { id: 2, name: 'Ben' }]
const names = pluck(users, 'name')          // string[]
const res: ApiResponse<typeof users> = { data: users, error: null }
console.log(first(names), res.data.length)
```

5. Utility types



Think of utility types like photo filters

Utility	Result
Partial<T>	every property optional (PATCH bodies)
Required<T>	every property required
Readonly<T>	every property readonly
Pick<T, "a" | "b">	only those keys
Omit<T, "password">	all keys except those
Record<K, V>	an object with keys K and values V
ReturnType<typeof fn>	the return type of a function
Awaited<P>	the value inside a promise

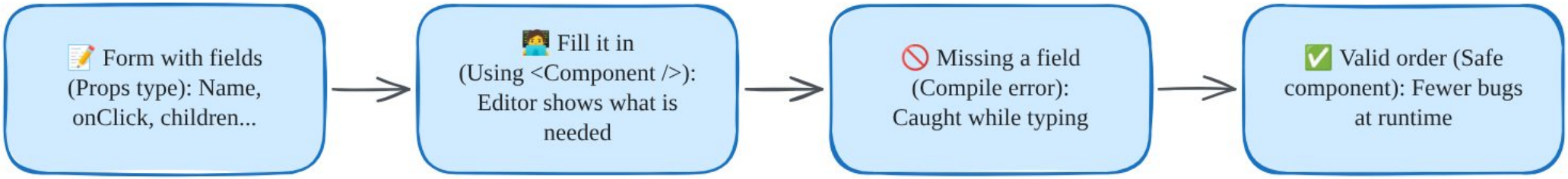
Typical API types

```
interface User { id: number; name: string; email: string; password: string }

type PublicUser = Omit<User, 'password'>
type UpdateUser = Partial<Pick<User, 'name' | 'email'>>
type UsersById = Record<number, PublicUser>

// keyof, typeof, and mapped types
type Flags<T> = { [K in keyof T]: boolean }
const config = { port: 3000, host: 'localhost' }
type Config = typeof config                    // { port: number; host: string }
```

6. TypeScript with React



Think of typed props like an order form with required boxes

Props, state and events

```
type ButtonProps = {
  label: string
  variant?: 'primary' | 'ghost'
  onClick: (event: React.MouseEvent<HTMLButtonElement>) => void
} & React.ComponentProps<'button'>

function Button({ label, variant = 'primary', ...rest }: ButtonProps) {
  return <button className={variant} {...rest}>{label}</button>
}

const [user, setUser] = useState<User | null>(null)
const inputRef = useRef<HTMLInputElement>(null)
```

Revise

Types are erased at runtime → validate external data. interface for object shapes, type for unions.
Narrow unknown, never use any. Generics keep types linked; Partial / Pick / Omit / Record cover most API types.

Rapid-fire Q&A (1/2)

interface vs type?

interface describes object shapes, can be extended and merged. type can also express unions, tuples and computed types but can't be re-opened.

any vs unknown vs never?

any disables checking; unknown accepts anything but must be narrowed before use; never is a value that cannot exist.

What are generics?

Type parameters that let one function or type work with many types while keeping input and output types linked, e.g. first<T>(items: T[]): T.

What is a discriminated union?

A union whose members share a literal field (like status). Switching on that field narrows to the right member.

Does TypeScript check types at runtime?

No. Types are erased when compiling to JavaScript, so data from APIs must be validated at runtime (e.g. with Zod).

Partial, Pick and Omit?

Partial makes all properties optional; Pick keeps chosen keys; Omit removes chosen keys.

Rapid-fire Q&A (2/2)

What does keyof do?

Produces a union of an object type's keys, e.g. keyof { a: 1; b: 2 } is "a" | "b".

How do you make a switch exhaustive?

Assign the value to a variable of type never in the default branch; the compiler errors if a case is missing.

What is a type guard?

A check that narrows a type — typeof, instanceof, in, or a function returning "x is Type".